

Documentation technique

BailPilote

A propos de cette documentation :

Cette documentation technique a pour but d'aider les développeurs travaillant sur l'application à la :

- développer,
- tester, et
- maintenir,

de manière collaborative et efficace.

Cette documentation doit être un outil facile et rapide à utiliser. Pour atteindre cet objectif, elle devra être exhaustive, tout en restant concise.

C'est-à-dire que :

- Tous les sujets devront être abordés (de la méthodologie de travail jusqu'aux fonctionnalités).
- On documentera de manière la plus courte et la plus explicite possible (on préférera les schémas et tableaux aux longs textes explicatifs).

Mise à jour

19/05/2026 : version 1.0

Table des matières

I)	Rappel sur le fonctionnement de l'application.....	3
1)	Description de l'application.....	3
2)	Utilisateurs de l'application et use cases.....	3
II)	Diagramme de classes et modèle physique de données.....	7
III)	Méthode de travail.....	8
1)	Méthodologie Scrum.....	8
2)	GitFlow.....	10
3)	Campagne de tests.....	12

I) Rappel sur le fonctionnement de l'application

1) Description de l'application

BailPilote est une application de gestion immobilière créée pour les propriétaires-bailleurs qui souhaitent gérer leur bien seul sans recourir aux services d'une agence immobilière.

En automatisant les tâches de gestion locative, BailPilote permet aux bailleurs de reprendre la main sur tous les aspects de leurs locations, sans dépense de temps et d'énergie supplémentaires par rapport à une agence classique et surtout sans les coûts financiers inhérents à celle-ci.

2) Utilisateurs de l'application et use cases

Les utilisateurs de l'application

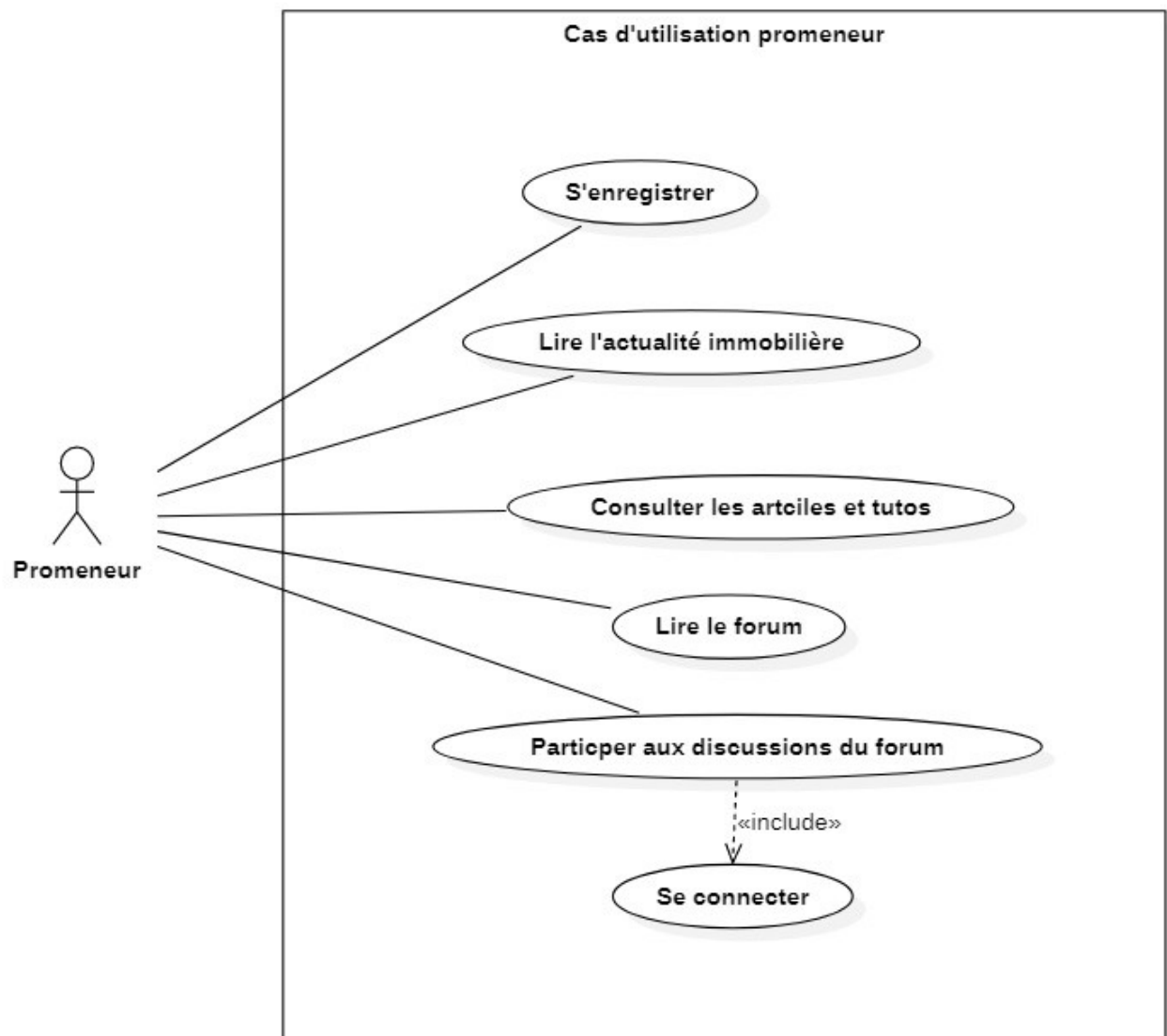
 (🔑 ?) Promeneur	Le promeneur est un utilisateur qui peut consulter anonymement et gratuitement le contenu éditorial et le forum. Il doit s'enregistrer et se connecter pour participer aux discussions du forum.
 🔑 Bailleur	Le promeneur devient bailleur lorsqu'il a souscrit un abonnement payant pour profiter des fonctionnalités de gestion immobilière locative.
 🔑 Locataire	Le locataire est invité par le bailleur à valider son compte pour profiter de son espace locataire qui lui facilitera ses démarches et la communication avec le bailleur.
 🔑 Super admin	Le super admin est un employé de BailPilote. Il possède tous les droits d'administration du site pour gérer les autres admins et les bailleurs.
 🔑 Juriste (< Super Admin)	Le juriste est un employé de BailPilote dont le profil a été créé par le super admin. Il a accès aux fonctionnalités éditoriales pour créer du contenu et peut répondre aux questions juridiques du locataire et du bailleur.
 🔑 Editorialiste (< Juriste)	L' éditorialiste est un employé de BailPilote dont le profil a été créé par le super admin. Il a accès aux fonctionnalités éditoriales pour créer du contenu.

Les diagrammes de cas d'utilisation

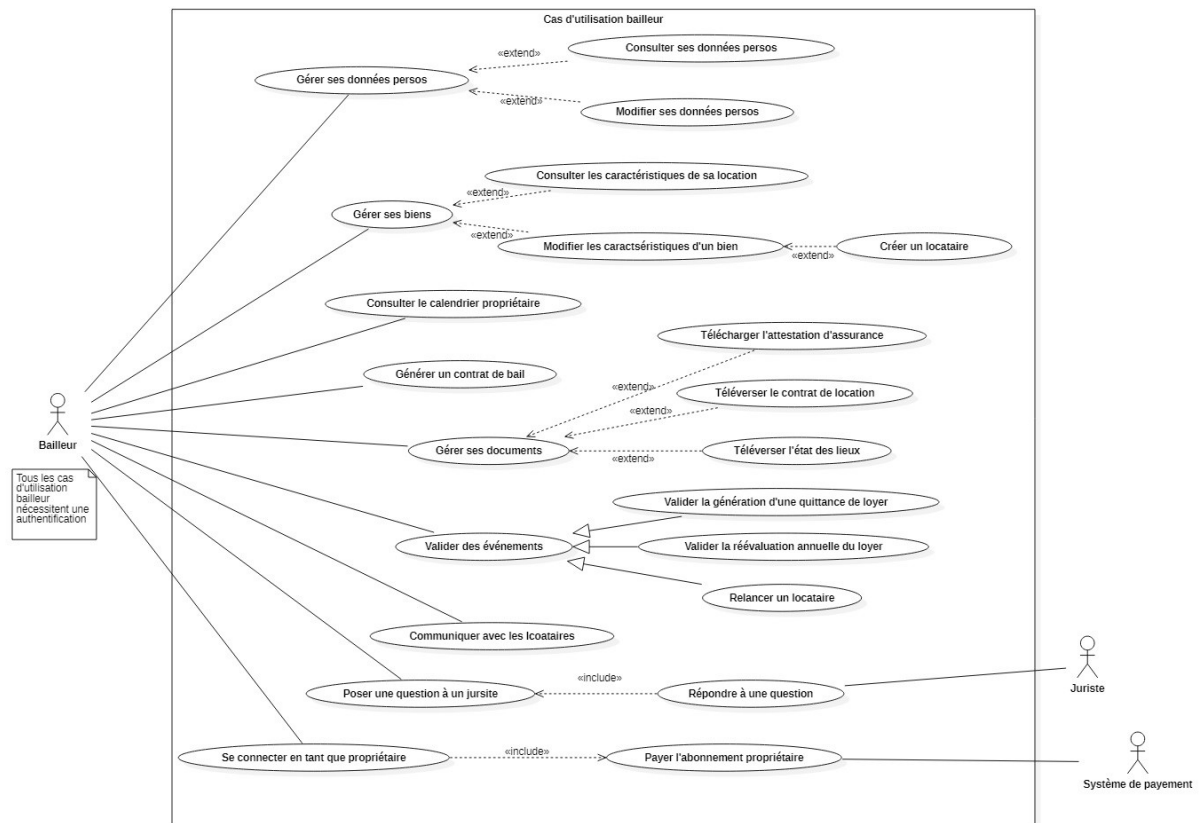
Travaillant en scrum, les besoins utilisateur sont au centre du projet. C'est donc assez naturellement que le découpage par lot de l'application s'est fait en fonction des besoins de chaque utilisateur : Promeneur, Bailleur, Locataire et Admin.

Les besoins de chaque utilisateur sont représentés dans les diagrammes de cas d'utilisation ci-dessous :

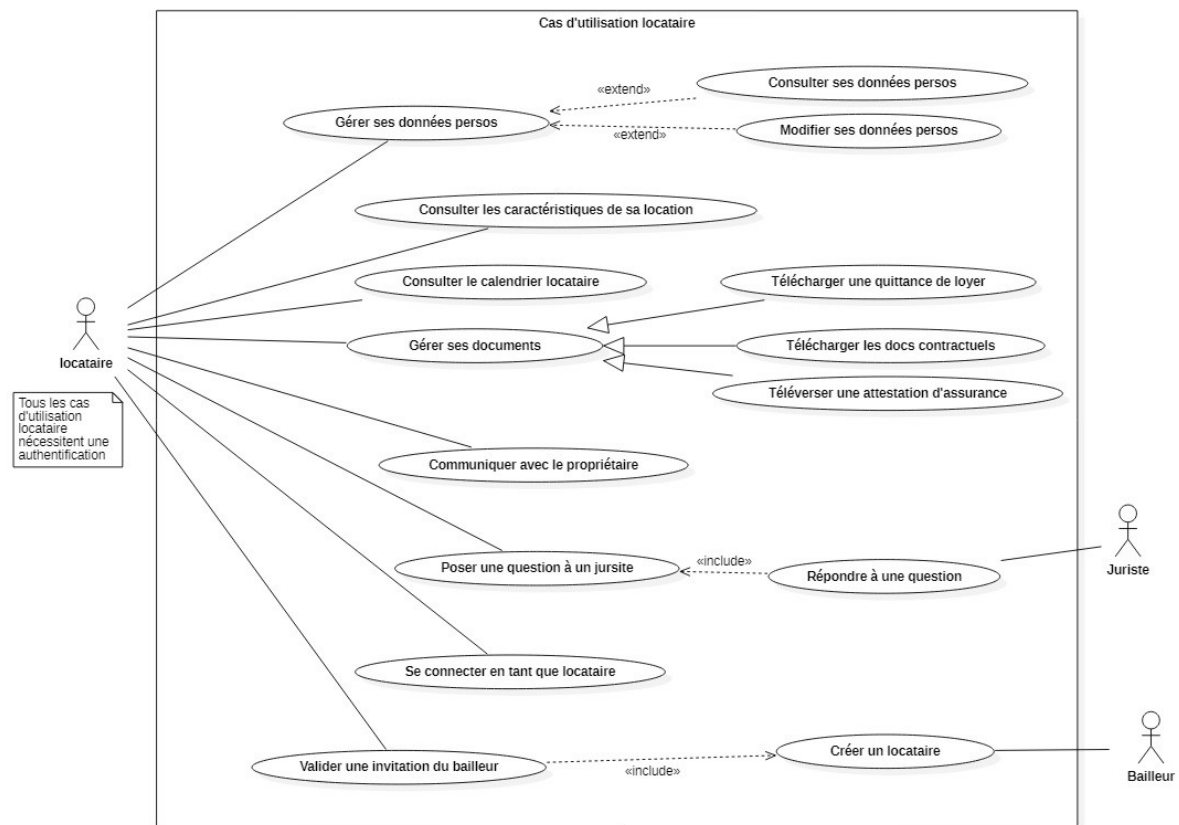
Lot n°1 : Vitrine de l'application



Lot n°2 : Cœur de l'application – espace bailleur

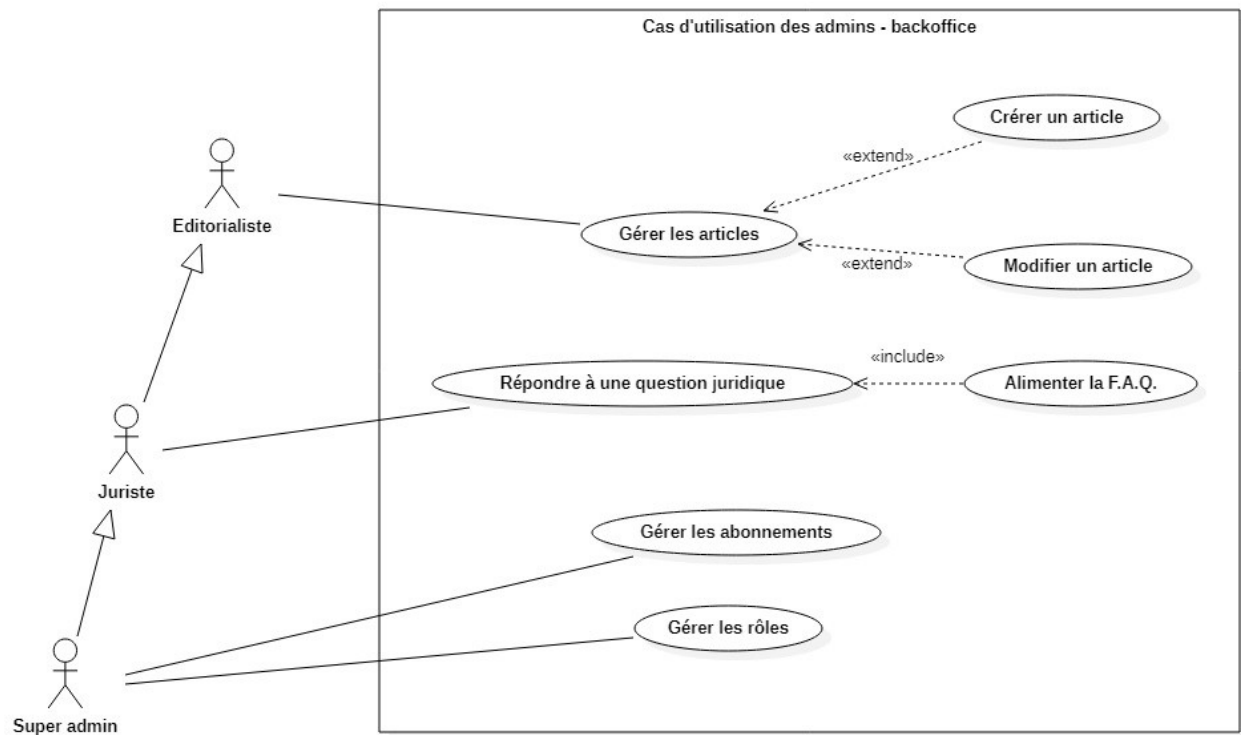


Lot n°3 : Cœur de l'application – espace locataire



Lot n°4 : Interface d'administration de l'application

Le Super Admin, le Juriste et l'Editorialiste sont tous des admins.



II) Diagramme de classes et modèle physique de données

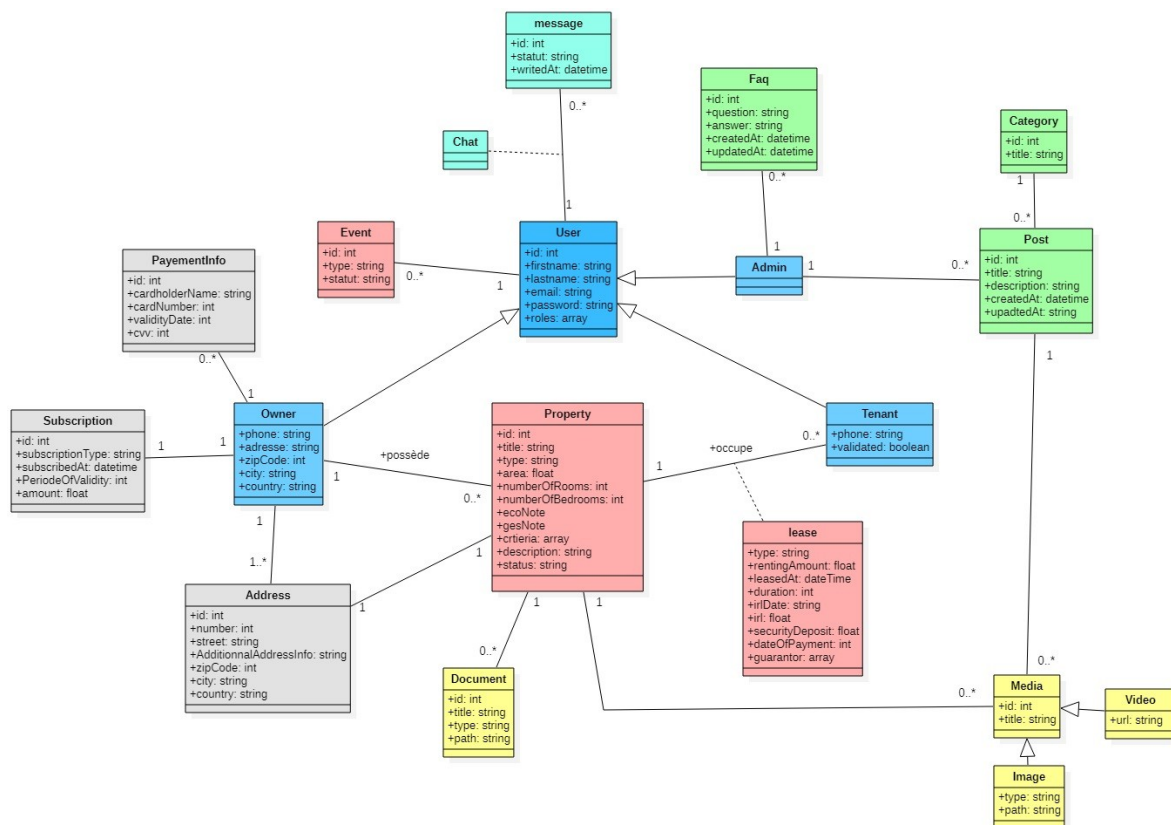
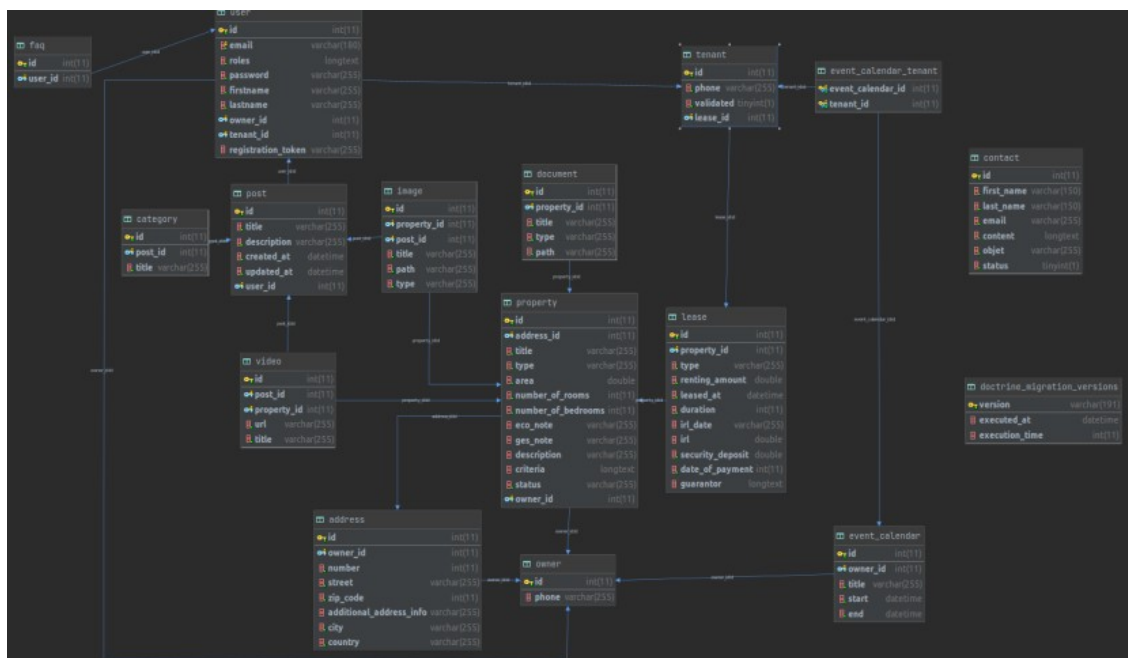


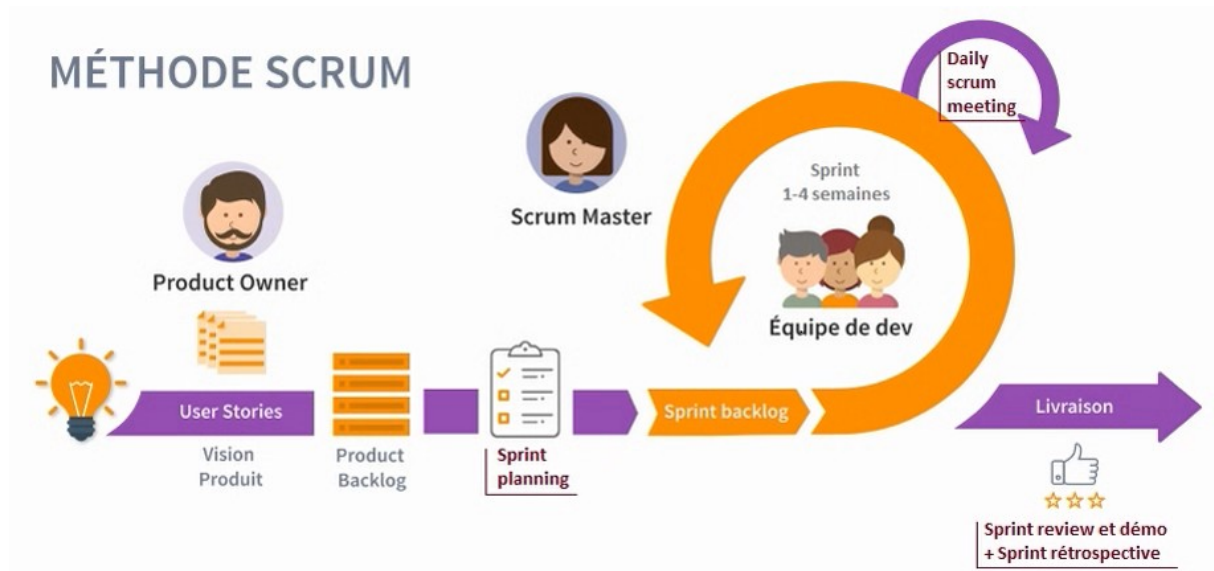
Diagramme de classes entités



Modèle physique de données

III) Méthode de travail

1) Méthodologie scrum



Les principes de la méthode Scrum

La priorité numéro 1 est de satisfaire le client grâce à une livraison rapide et continue d'un prototype opérationnel pour l'utilisateur, afin qu'il puisse l'évaluer. C'est une approche dynamique et participative qui implique un engagement de la scrum team à réorienter le projet au fil de son avancement.

Les acteurs de la méthode Scrum

La **scrum team** est composée :

- D'un **scrum master** qui est le leader et le garant de la mise en place de la méthode scrum et de l'efficacité de la scrum team.
- D'un **Product Owner** qui détient la vision du produit à réaliser. Il représente les utilisateurs et le commanditaire.
- D'une **équipe de développeurs** qui transforme une sélection du Product Backlog en un incrément (= résultat utilisable par l'utilisateur).



Les événements de la méthode Scrum

Le projet est divisé en **sprint** (de 1 à 4 semaines chacun).

Chaque sprint est ponctué de 4 événements :

- Le **sprint planning** : c'est une réunion qui lance le sprint. Son but est de définir ses objectifs et d'organiser son déroulement (2h maximum par semaine de sprint).

- Le **Daily Scrum meeting** : c'est une réunion quotidienne (5-10 minutes) ayant pour but de maintenir le lien dans l'équipe et de favoriser la collaboration et l'entraide en son sein.
- Le **sprint review** : c'est une réunion de fin de sprint ayant pour but de présenter les incréments réalisés durant le sprint et de réfléchir à son déroulement afin de réorienter si besoin le produit (2h maximum par semaine de sprint).
- Le **sprint retrospective** : c'est une réunion ayant pour but d'analyser le sprint terminé, afin de mettre en place des stratégies pour améliorer les suivants (45 minutes par semaine de sprint).

La définition du « Done »

Un incrément est une tâche entièrement terminée produisant une fonctionnalité utilisable par l'utilisateur.

Une tâche est dite « done » et du sprint backlog, si et seulement si, la checklist ci-contre est entièrement cochée.

Définition du Done

- ☐ Développement terminé
- ☐ Test unitaire réalisé
- ☐ Code revu
- ☐ Code documenté
- ☐ Campagne de test validée

Outil de gestion de projet

Nous utilisons Jira comme outil de gestion de projet.

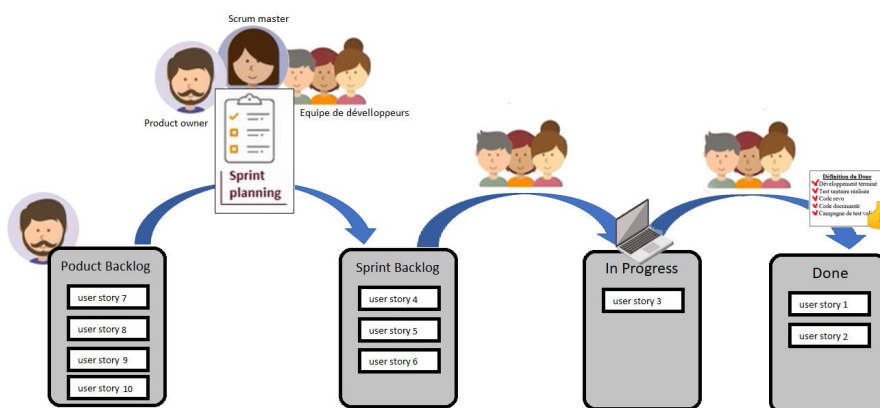
Au début du projet, le *product owner* place toutes les user stories dans la liste « **Product Backlog** ».

Par souci de lisibilité, le « product backlog » a été divisé en lots.

Lors du sprint planning, la *scrum team* déplace collégialement dans la liste « **Sprint Backlog** » les user stories qui seront développées lors du sprint à venir.

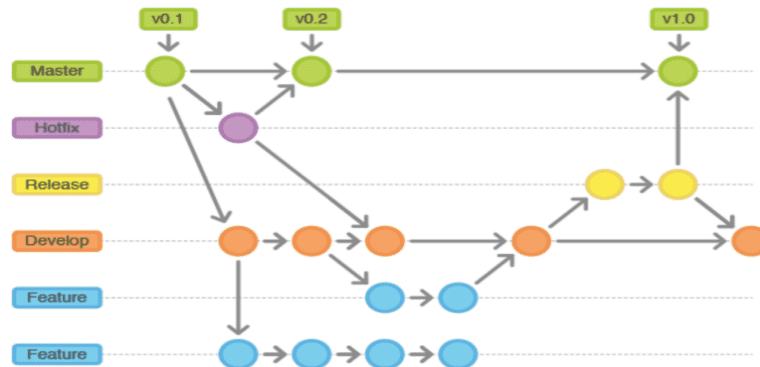
Lors du développement, lorsqu'une user stories est prise en charges par un *développeur*, il la déplace dans la liste « **In Progress** ».

Une fois qu'une user stories a été développée et satisfait aux critères de la définition du « Done », elle est placée dans la liste « **Done** ».



2) GitFlow

Les branches du GitFlow



La branche master : c'est la branche de production.

Chaque merge sur cette branche donnera lieu à une nouvelle version :

- Mineure pour les merges de hotfix
- Majeure pour les merges de release

Ce qui sera mergé sur cette branche devra faire preuve de la plus grande attention et sera de la responsabilité unique du lead développeur.

La branche dev : C'est la branche de développement qui recevra les fonctionnalités achevées, revues, documentées, testées et approuvées par le lead développeur.

Les branches feature : Ce sont des branches éphémères.

Chacune sera consacrée au développement d'une fonctionnalité précise. Une fois la fonctionnalité achevée, revue, documentée et testée, le développeur qui l'a traitée proposera un merge de sa branche sur la branche dev au lead dev.

- Si le merge est accepté et réalisé, la branche feature est alors supprimée.
- Si le merge est refusé, le lead dev indique les points à améliorer par le développeur avant qu'il puisse la proposer à nouveau au merge.

La branche release : C'est la branche qui permet de développer et tester une nouvelle version de l'application avant sa mise en production.

La branche hotfix : C'est une branche destinée à résoudre des bugs de la branche master.

Création d'une nouvelle fonctionnalité

Création d'une branche

Les développeurs travaillent sur les branches feature en partant de la branche dev.

- Créer une nouvelle branche en se plaçant dessus à partir de la branche « dev » :
`git checkout -b feat/myNewFeature dev`
- Faire un premier commit vide pour activer cette branche :
`git commit -m « new branche feat/myNewFeature » --allow-empty`

Développement de la fonctionnalité

Développer la fonctionnalité :

- en respectant les standards de code (voir [paragraphe III-4](#) : standard de code)
- en écrivant OBLIGATOIREMENT les tests unitaires et fonctionnels nécessaires (voir [paragraphe III-3](#) : campagne de tests)
- en commitant les changements importants (voir [paragraphe III-4](#) : règle de commit)
 - `git add <file name>` #ajouter le fichier au stage
 - `git commit` #commiter les changements
- En documentant et commentant votre code (voir [paragraphe III-4](#): commentaires)

Revue de la fonctionnalité

- Vérifier les standards de code avec code sniffer etc.
- Vérifier que vos tests et les autres passent correctement (voir [paragraphe III-3](#) : campagne de tests)
- Vérifier la qualité
- Vérifier la performance du code (requête sql, etc.)

Merge de la fonctionnalité sur la branche dev

- Envoyer votre code sur le dépôt distant github
 - `git push origin feat/myNewFeature`
- Proposer une pull-request au lead dev
 - o Si la pull-request est refusée, effectuer les modifications attendues par le lead développeur.
 - o Si elle est acceptée, le lead développeur mergera la pull-request.
- Si le lead dev vous demande de merger la pull-request à sa place
 - `git checkout dev` #se placer sur la branche dev
 - `git rebase feat/myNewFeature` #linéariser l'historique
 - `git merge feat/myNewFeature` (si nécessaire) #merger votre branche sur la branche dev
 - ➔ Résoudre les conflits éventuels
 - `git branch -d feat/myNewFeature` #supprimer votre branche

Particularité des hotfixs et des releases :

Pour résoudre une hotfix ou créer une realease, l'esprit et la méthode sont identiques avec quelques variations listées dans le tableau ci-dessous.

	hotfix	release
Création de la branche à partir de	master	dev
Merge de la branche sur	dev et master	dev et master
Ajouter un tag de version	git tag V.1.2	git tag V.2.0

Règle de commit

A moins d'un commit évident, on évitera la commande `git commit -m « mon message »` pour commiter.

On préférera la commande `git commit`.

Une fois dans l'invite de commande, on écrira :

- Un titre explicite, pas trop long (49 caractères maximum)
- Un texte de description (pertinent et clair)

3) Campagne de tests

On ne recherche pas 100% de couverture de code par les tests.

Par contre, DOIVENT ÊTRE TESTES :

- L'intégralité du code métier.
- Les points critiques de l'application
- Le maximum de cas limite

Pour tester l'API, on utilise PHPUnit dans le dossier \tests

Avant de lancer les tests, préparer la BDD de test :

Composer prepareDB-test

Vérifier que tous les tests (ceux existants et les vôtres) passent au vert.

Pour cela, lancer la commande : *php bin/phpunit*

PhpUnit propose un outil de contrôle de la couverture de code.

Dans la console : *php bin/phpunit --coverage-html public/test-coverage*

Dans le navigateur : *https://127.0.0.1:8000/test-coverage*

	Code Coverage								
	Lines			Functions and Methods			Classes and Traits		
Total		100.00%	264 / 264		100.00%	69 / 69		100.00%	18 / 18
📁 Controller		100.00%	64 / 64		100.00%	11 / 11		100.00%	4 / 4
📁 DataFixtures		100.00%	42 / 42		100.00%	5 / 5		100.00%	2 / 2
📁 Entity		100.00%	50 / 50		100.00%	30 / 30		100.00%	2 / 2
📁 Form		100.00%	18 / 18		100.00%	2 / 2		100.00%	2 / 2
📁 Repository		100.00%	10 / 10		100.00%	3 / 3		100.00%	2 / 2
📁 Security		100.00%	45 / 45		100.00%	11 / 11		100.00%	3 / 3
📁 Service		100.00%	35 / 35		100.00%	7 / 7		100.00%	3 / 3